Tema Finale ISS 2026 – Sprint 2

Edoardo Buttazzi

1 Introduction

1.1 Risultati Sprint 1

Lo Sprint 1 ha realizzato un prototipo eseguibile e distribuito del ciclo completo di carico, strutturato su quattro attori QAK: `cargoservice`, `cargorobot`, `ioportservice` e `sensorservice`.

Il prototipo sviluppato gestisce l'accettazione o il rifiuto delle richieste in base alla disponibilità degli slot (`slot1`–`slot4`), della IOPort e dello stato del servizio, coordinando il trasporto del container attraverso l'area di marcatura (`slot5`) fino allo slot riservato e il successivo rientro in `HOME` tramite il servizio esterno `RobotSmart26`.

Nello Sprint 1 l'interazione con il pulsante, il display e il flusso di distanze del sonar è stata temporaneamente effettuata mediante componenti simulati.

1.2 Goal dello Sprint 2

Questo sprint completa l'interazione con l'ambiente fisico e con il cliente attraverso tre interventi:

1. realizzazione della IOPort come web-GUI.
2. integrazione di sonar e LED sul Pico W.
3. gestione dei casi di fuori servizio.

Il documento distingue quindi i comportamenti richiesti, i problemi architetturali e le scelte di realizzazione. I test coprono le nuove funzionalità.

2 Requirement analysis

Questo capitolo precisa i comportamenti richiesti al sistema, riprendendo i vincoli consolidati nello Sprint 1 e formalizzando i chiarimenti concordati con il committente per i dispositivi fisici e la gestione delle anomalie.

2.1 Requisiti dello sprint e vincoli ereditati

Si confermano e completano i requisiti definiti negli sprint precedenti:

1. **Rilevamento del container:** Il flusso di misurazioni del sonar alla IOPort deve consentire il riconoscimento del container quando misura $D < D_{FREE}/2$ per 3s continui, confermati entro una finestra massima di 30s dall'accettazione della richiesta.
2. **Rilevamento del fuori servizio:** Se la distanza misurata dal sonar supera D_{FREE} per almeno 3 secondi di fila, il sistema segnala lo stato di *Out of service* e rinvia le nuove richieste notificando `retrylater`.
3. **LED:** Durante l'intero ciclo di carico in cui il sistema è *engaged* (dall'accettazione della richiesta fino al rientro definitivo del robot in HOME), il LED deve lampeggiare, mentre a sistema *disengaged* il LED deve essere spento.

4. **Web-GUI:** La Web-GUI della IOPort include un pulsante dedicato per abilitare l'invio delle richieste di carico da parte dell'utente e mostrare la risposta ricevuta (load_accepted(SLOT), retrylater oppure load_refused per stiva piena). Il display deve inoltre visualizzare in tempo reale lo stato degli slot (1–4) e mostrare *Service working* oppure *Out of service*.

5. **Regole di coordinamento e stiva:** Il servizio gestisce un solo caricamento alla volta. All'accettazione viene prenotato lo slot libero con indice minore (da `slot1` a `slot4`). La prenotazione decade se scade il timeout di arrivo del container, mentre lo slot diventa `OCCUPIED` unicamente a deposito finale completato.

2.2 Ulteriori chiarimenti concordati con il committente

Per chiudere i casi limite legati all'integrazione dei dispositivi e alle anomalie in corso d'opera, sono state concordate e fissate le seguenti specifiche:

– **A1 - Comportamento del robot a seguito di rilevamento guasti durante un carico:** Se l'anomalia ($D > D_{FREE}$ per 3s continui) viene rilevata mentre il robot è in moto, esso non viene arrestato istantaneamente, ma completa lo spostamento e l'eventuale deposito attivo, mantenendo il sistema *engaged* e il LED lampeggiante. Solo a fine tratta il servizio entra in *Out of service*, spegne il LED e impedisce ulteriori movimenti, preservando lo stato della stiva e le prenotazioni per la successiva manutenzione esterna.

– **A2 - Mancanza delle misure del sonar:**
La caduta del collegamento con il microcontrollore non deve essere interpretato come porta libera, né equiparate ad un'anomalia fisica ($D > D_{FREE}$). In assenza di dati recenti e determinabili sullo stato della IOPort, il coordinatore sospende momentaneamente l'accettazione dei carichi rispondendo `retrylater`, senza innescare la condizione permanente di *Out of service*. La ricezione di misure nuovamente stabili e valide riabilita automaticamente il normale flusso operativo.

– **A3 - Calibrazione e continuità temporale delle osservazioni:**
La distanza a porta libera (D_{FREE}) è una costante determinata durante la configurazione e calibrazione iniziale dell'impianto. Il vincolo di conferma sui 3s continui richiede un flusso regolare di campioni: la rilevazione di pause temporali o buchi anomali tra letture successive determina l'azzeramento immediato del contatore.

3 Problem Analysis

L'analisi si concentra sull'integrazione di due elementi chiave: l'hardware fisico e l'interfaccia browser.

3.1 Gestione dei dispositivi e interpretazione dei dati

Come già detto nello Sprint precedente, per disaccoppiare l'hardware dalla logica di business, il sistema acquisisce i dati grezzi dei sensori e ne delega l'interpretazione al livello applicativo. In questo modo, il coordinatore non deve gestire le singole letture, ma riceve soltanto informazioni aggregate ad alto livello (come presenza del carico, disponibilità della porta o anomalie).

Perciò, come già anticipato l'elaborazione temporale delle letture è delegata a `sensorservice`. Separare il controllo dei sensori dal coordinamento del carico permette inoltre di riutilizzare lo stesso componente sia nei test simulati sia con l'hardware reale.

Il monitoraggio è continuo, sia a riposo sia durante la movimentazione:

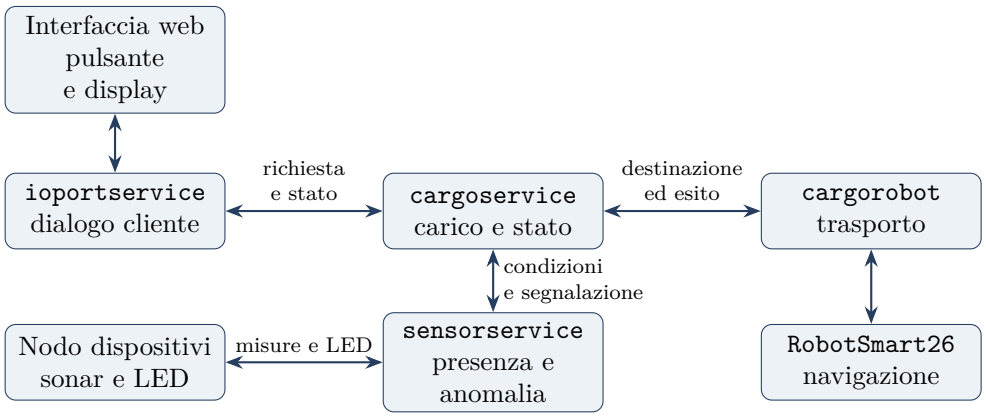
- se una porta risulta già occupata prima di una nuova richiesta, `cargoservice` viene avvisato subito.
- quando un carico è stato accettato ed è in attesa, la presenza del container sblocca il prelievo solo se il servizio è regolarmente attivo e il tempo limite non è scaduto.
- la segnalazione di anomalia ha invece la priorità assoluta: porta subito il coordinatore in stato di fuori servizio, bloccando l'accettazione di nuovi ordini.

Infine, lo stato del sensore non può basarsi semplicemente sull'ultimo valore registrato in memoria: serve verificare anche la validità temporale della lettura per capire se il dato sia ancora affidabile o se la comunicazione si sia interrotta.

3.2 Architettura logica

La tabella riassume le responsabilità elaborate negli sprint precedenti.

Componente logico	**Responsabilità**
`cargoservice`	Decide l'accettazione, coordina il ciclo, conserva stato e barcode e applica la politica di fuori servizio.
`sensorservice`	Interpreta le distanze, conferma presenza e anomalia, segnala l'attendibilità del rilevamento e inoltra l'intenzione di segnalazione luminosa.
`ioportservice`	Collega il dialogo del cliente al servizio e rende disponibili risposta alla richiesta e stato corrente da visualizzare.
`cargorobot`	Esegue le richieste di trasporto attraverso il navigatore e comunica gli esiti, preservando le responsabilità dello Sprint 1.
Sensori e LED	Acquisiscono la distanza e realizzano la segnalazione luminosa.
Interfaccia web	Raccoglie l'azione del cliente e presenta risposte, stato stiva, e disponibilità del collegamento.
`RobotSmart26`	Fornisce la navigazione nell'ambiente virtuale già adottata nello Sprint 1.



3.3 Comunicazione dell'architettura finale

La tabella distingue le richieste che richiedono un esito dalle notifiche e dai comandi asincroni.

Messaggio	Scambio	Funzione
`load_request`	Request/Reply: `ioportservice` verso `cargoservice`.	Restituisce `load_accepted(SLOT)`, `retrylater` oppure `load_refused`. La IOPort riporta l'esito al richiedente.
`sensor_state`	Dispatch: `sensorservice` verso `cargoservice`.	Comunica `free`, `present` o `anomaly`; il coordinatore ne decide gli effetti sul carico.
`sensor_unavailable`	Dispatch: `sensorservice` verso `cargoservice`.	Segnala dati non attendibili e impedisce nuove accettazioni senza causare un guasto permanente.
`led_command`	Dispatch: `cargoservice` verso `sensorservice`.	Richiede `blink` oppure `off`, da inoltrare al dispositivo.
`display_update`	Dispatch: `cargoservice` verso `ioportservice`.	Notifica lo stato applicativo aggiornato, comprendente fase, operatività, slot e barcode.
`reach_target`	Request/Reply: `cargoservice` verso `cargorobot`.	Richiede una destinazione logica e riceve `target_reached` oppure `target_failed`.
`get_ioport_status` `get_hold_status`	Request/Reply: adattatore web verso IOPort, quindi IOPort verso coordinatore.	Recupera una fotografia corrente dello stato, mantenendo `cargoservice` come unico proprietario dell'inventario.

3.4 Punti principali da elaborare nella progettazione

Questa organizzazione logica fa emergere tre criticità:

P1 – Collegamento al nodo dispositivi.
Trasportare misure e comandi su una connessione adatta a un nodo con risorse limitate, gestendo la sua disponibilità senza introdurre nel dispositivo il modello del carico.

P2 – Interpretazione e attuazione concorrenti.
Realizzare in `sensorservice` la conferma temporale e il controllo dell'attendibilità, mantenendo l'acquisizione e il lampeggio indipendenti dalla sequenza delle tratte.

P3 – Confine web.
Collegare browser e `ioportservice`, conservando la correlazione delle richieste e rendendo il display aggiornabile per notifica, anche dopo una riconnessione.

4 Test Plans

I controlli definiti nello Sprint 1 restano validi, mentre i nuovi test si concentrano su integrazione dei dispositivi, interfaccia web e gestione delle anomalie. A partire da uno stato iniziale noto di stiva e servizio, si verificano le risposte al client, lo stato degli slot, l'operatività e i movimenti del robot (inclusa la verifica del LED sull'hardware fisico).

TP1 – Ciclo completo con GUI, sonar e LED.
Con stiva vuota, porta libera e servizio operativo, il cliente invia una richiesta dalla web-GUI. Il sistema deve accettarla e riservare il primo slot disponibile. Si presenta quindi il container al sonar: il robot parte soltanto dopo la conferma della presenza entro il tempo previsto. Durante il ciclo si controllano il lampeggio del LED, il passaggio dello slot da prenotato a occupato e l'aggiornamento del display. Al ritorno a HOME il servizio torna disponibile e il LED si spegne.

TP2 – Mancato arrivo del container.
Dopo l'accettazione, si lascia libera la porta fino alla scadenza dell'attesa. Il sistema deve annullare la prenotazione, restare fermo, aggiornare la GUI e spegnere il LED. Una conferma di presenza successiva alla scadenza non deve riattivare il carico annullato.

TP3 – Fuori servizio causato dal sonar.
Si prova prima una breve sequenza sopra soglia, che non deve confermare l'anomalia, poi una sequenza continua della durata richiesta. Il servizio deve segnalare `Out of service` e rispondere `retrylater` a una nuova richiesta. Il ritorno a misure regolari non deve ripristinarlo automaticamente. Si ripete la prova durante una tratta: il robot conclude quella già avviata, senza iniziarne altre. Prenotazioni ed effetti confermati restano conservati e il LED si spegne alla sospensione.

TP4 – Risposte e visualizzazione della IOPort.
Oltre all'accettazione verificata in TP1, si invia una richiesta con servizio impegnato o porta occupata e una con stiva piena. La GUI deve mostrare rispettivamente `retrylater` e il rifiuto del carico, senza prenotazioni aggiuntive. Riaprendo la pagina si deve ottenere lo stato corrente della stiva e del servizio.

TP5 – Perdita delle misure del sonar.
Si interrompe il flusso delle letture. Superato l'intervallo di attendibilità configurato, la porta deve risultare non determinabile e le nuove richieste non devono essere accettate. La perdita dei dati non deve essere interpretata come porta libera o presenza confermata. Alla ripresa delle misure si richiede una nuova osservazione valida, senza usare il tempo trascorso durante l'interruzione per completare una conferma.

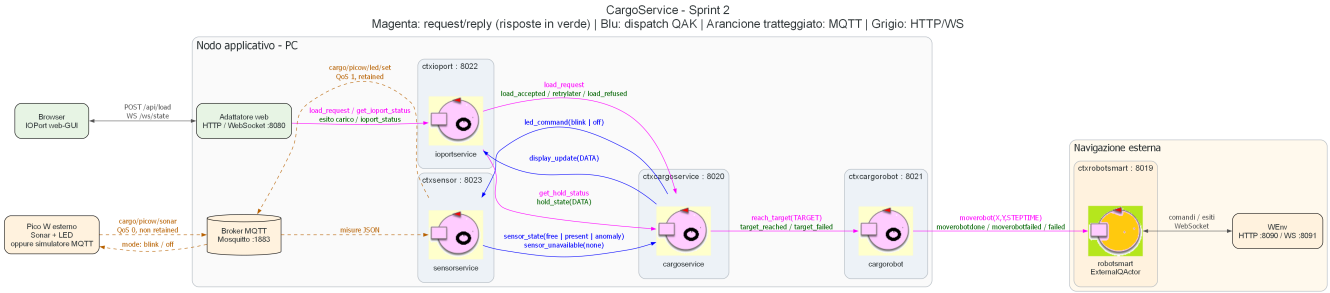
5 Project

La progettazione realizza i punti individuati nell'analisi: collegamento al nodo dispositivi, interpretazione delle letture e confine web. Le responsabilità degli attori restano quelle definite nei capitoli precedenti.

5.1 Architettura finale

I quattro attori mantengono contesti QAK separati, eseguibili sullo stesso elaboratore. Il Pico W esegue il firmware di acquisizione e attuazione in MicroPython. Il collegamento fisico usa un broker MQTT, mentre un adattatore web collega il browser a `ioportservice`.

Unità	Accesso
`ctxcargoservice`	TCP 8020
`ctxcargorobot`	TCP 8021
`ctxioport`	TCP 8022
`ctxsensor`	TCP 8023
`RobotSmart26`	TCP 8019
Broker MQTT	TCP 1883
Adattatore web	HTTP/WS 8080
Pico W	Client MQTT



5.2 P1 - Collegamento al nodo dei dispositivi

Il collegamento al nodo dispositivi deve trasportare un flusso periodico di misure e, nella direzione opposta, comandi luminosi. Il Pico W deve conoscere il punto di accesso alla comunicazione, senza dipendere dall'indirizzo del componente applicativo che interpreta le letture.

Si adotta MQTT perché separa produttore e consumatore attraverso topic pubblicati su un broker. Lo stesso contratto può essere utilizzato dal dispositivo fisico e dal simulatore, senza modificare gli attori.

Un collegamento TCP diretto sarebbe possibile, ma richiederebbe di gestire esplicitamente la connessione fra il dispositivo e il relativo interlocutore, oltre al framing dei messaggi. Anche HTTP potrebbe trasportare le misure, ma richiederebbe una soluzione aggiuntiva per consegnare i comandi al dispositivo. MQTT offre invece entrambi i versi di comunicazione mediante il medesimo meccanismo publish/subscribe.

Il costo accettato è la presenza di un broker da avviare e configurare, insieme alla necessità di distinguere la disponibilità del trasporto dall'attendibilità delle misure ricevute.

L'interscambio si articola su due topic principali:

- `cargo/picow/sonar`: dedicato allo streaming periodico delle misurazioni.
- `cargo/picow/led/set`: impiegato per impostare lo stato operativo del LED.

```
// Lettura sonar (cargo/picow/sonar)
{
  "session": "s1",
  "seq": 12,
  "distance": 40.0,
  "valid": true
}

// Comando LED (cargo/picow/led/set)
{
  "mode": "blink"
}
```

Per quanto riguarda i parametri di trasmissione, Le misure sonar usano QoS 0, accettando la perdita occasionale di campioni del flusso periodico, e non sono retained, per evitare che una nuova sottoscrizione riceva una vecchia misura come dato corrente.

Al contrario, i comandi per il LED richiedono garanzia di consegna e adottano perciò QoS 1. Poiché questa politica ammette la ricezione di messaggi duplicati (*at least once*), la logica di attuazione sul firmware è progettata per essere idempotente, affinchè l'impostazione di uno stato già attivo non generi nuovi task concorrenti né alteri il ciclo in corso.

5.3 P2 - Acquisizione, interpretazione e controllo del LED

Acquisizione e parametri. La configurazione iniziale comprende collegamenti del sonar, unità delle distanze e valore di D_{FREE}, misurato con la porta libera. Il riferimento rimane fisso durante un caricamento.

Come valori iniziali si propongono un campionamento ogni 100 ms e una massima discrepanza di 500 ms.

5.3.1 Funzionamento logico del rilevatore mediante FSM

Per capire cosa sta succedendo davanti alla porta non basta guardare una sola lettura del sonar: bisogna controllare come cambiano le misure nel tempo. Per questo motivo `sensorservice` tiene traccia dello stato delle osservazioni e comunica al coordinatore solo le informazioni stabili (porta libera, container arrivato, anomalia o dati assenti).

Gli stati del rilevatore. Questi cinque stati teorici descrivono il comportamento logico del rilevatore.
Nell'implementazione, `unknown e observing` corrispondono entrambi alla condizione `unavailable`: nel primo caso mancano dati affidabili, nel secondo la presenza o l'anomalia è ancora in fase di verifica. Entrambe le situazioni impediscono l'accettazione di nuovi carichi. Quando una nuova misura valida conferma il completamento dei tre secondi continui, la condizione diventa rispettivamente `present` oppure `anomaly`.

- `unknown`: all'avvio, oppure quando il sensore non invia dati o le misure non sono utilizzabili. Il sistema risponde con `sensor_unavailable`.

- `free`: la porta risulta libera. Viene segnalato `sensor_state(free)`.
- `observing`: il sensore ha iniziato a rilevare un container o una possibile anomalia, ma si sta ancora aspettando che passino i 3 secondi necessari per la conferma. In questo intervallo la porta non è considerata libera e viene segnalato `sensor_unavailable`.
- `present`: un container è rimasto fermo davanti al sonar per almeno 3 secondi continui. Viene segnalato `sensor_state(present)`.
- `anomaly`: la distanza misurata è rimasta sopra la soglia massima per almeno 3 secondi continui. Viene segnalato `sensor_state(anomaly)`.

Il sensore non invia messaggi continui: comunica quando rileva un cambio di stato, inviando poi semplici conferme periodiche per ricordare che è ancora attivo.

Tipi di misura. Ogni lettura valida viene catalogata in base alla distanza D rilevata:

- **Tipo Low ($D < D_{\text{FREE}}/2$):** possibile arrivo di un container.
- **Tipo Normal ($D_{\text{FREE}}/2 \leq D \leq D_{\text{FREE}}$):** porta libera senza ostacoli.
- **Tipo High ($D > D_{\text{FREE}}$):** possibile guasto o anomalia del sonar.

I valori che cadono esattamente sui limiti appartengono al tipo Normal e interrompono qualsiasi conteggio in corso.

Transizioni. La tabella seguente mostra come cambia lo stato del rilevatore in base alle nuove misure ricevute. Una misura è considerata continua se arriva entro mezzo secondo dalla precedente.

Stato corrente	Evento e condizione	Stato successivo
Qualsiasi	Arriva una misura di tipo Normal.	`free`
Qualsiasi	Arriva una misura di tipo Low o High che inizia un nuovo conteggio, cambia tipo oppure arriva dopo una pausa troppo lunga ($> 500\,\text{ms}$).	`observing`
`observing`	Continuano ad arrivare misure di tipo Low e sono trascorsi almeno 3 secondi consecutivi.	`present`
`observing`	Continuano ad arrivare misure di tipo High e sono trascorsi almeno 3 secondi consecutivi.	`anomaly`
`present`	Continuano ad arrivare regolarmente misure di tipo Low.	`present`
`anomaly`	Continuano ad arrivare regolarmente misure di tipo High.	`anomaly`
Qualsiasi	Lettura con valore non valido oppure perdita di connessione con il dispositivo.	`unknown`
Qualsiasi	Mancano dati validi da più di 2 secondi.	`unknown`

Il solo passare del tempo non cambia mai lo stato da `observing` a `present` o `anomaly`: serve sempre una nuova misura valida dello stesso tipo che verifichi il completamento dei 3 secondi. Se invece non arrivano dati per oltre 2 secondi, il sistema passa automaticamente a `unknown`.

Controllo del LED. Come già anticipato il controllo dello stato del led è gestito da `cargoservice`, che inoltra a `sensorservice` i messaggi ad alto livello `led_command(blink)`

oppure `led_command(off)`. Spetta quindi a `sensorservice` il compito di tradurre queste direttive nel formato atteso dal Raspberry Pi Pico W, il quale si occupa della gestione vera e propria sul pin fisico.

La segnalazione luminosa a intermittenza accompagna l'intera operazione di movimentazione del carico e rimane attiva fino a quando il robot non conclude la missione tornando nella posizione di HOME. Nell'eventualità in cui si verifichi un'anomalia o un guasto durante la navigazione lungo una tratta, il coordinatore non interrompe bruscamente la segnalazione, bensì preserva il lampeggio fino a quando non viene determinato l'esito definitivo dell'operazione, richiedendo lo spegnimento effettivo soltanto nel momento in cui il sistema passa nello stato di sospensione.

5.3.2 Gestione del fuori servizio nel coordinatore

La ricezione di `sensor_state(anomaly)` imposta subito `OUT_OF_SERVICE` e genera un aggiornamento per la GUI. Un successivo ritorno a misure regolari non ripristina automaticamente il servizio.

In assenza di spostamenti in corso, il ciclo si sospende immediatamente. Qualora invece il robot sia già in movimento, `cargoservice` ne attende la conclusione naturale senza inviare comandi di arresto forzato. Una volta completata tratta e l'eventuale deposito del carico, l'attività viene arrestata evitando l'avvio della transizione successiva.

La sospensione mantiene prenotazioni, barcode ed effetti confermati. Le attese di arrivo e di marcatura vengono disattivate, eventuali timeout già in coda non devono liberare lo slot o far proseguire il carico.

5.4 P3 - Collegamento della web-GUI

ioportservice. Durante la progettazione dell'interfaccia si è valutata l'ipotesi di rimuovere `ioportservice`, collegando l'adattatore web direttamente a `cargoservice`.

Eliminare questo attore intermedio avrebbe semplificato il sistema: un salto TCP in meno (eliminando la porta 8022), tempi di risposta leggermente inferiori e una JVM in meno da avviare e gestire negli script di deployment (`ctxioport`).

Tuttavia si è scelto di mantenerlo per conservare un punto di accesso alla IOPort indipendente dalla tecnologia web e coerente con lo Sprint 1. Preservare l'attore permette inoltre a `cargoservice` di rimanere indipendente rispetto al mezzo usato dal cliente, lasciando la logica di business invariata anche a fronte di future modifiche al frontend.

Integrazione HTTP/WebSocket e adattatore web. Per collegare la GUI al sistema è stato sviluppato un adattatore in Python che espone endpoint HTTP e WebSocket, traducendo i messaggi JSON del frontend nei protocolli TCP/QAK usati dagli attori.

Si è scartata l'alternativa di connettere direttamente il browser al broker tramite MQTT-over-WebSocket: pur evitando un processo aggiuntivo, avrebbe costretto il frontend a gestire topic, payload interni e la correlazione asincrona tra richieste e risposte. L'adattatore permette invece di mantenere la pagina web completamente disaccoppiata dall'infrastruttura QAK.

L'interazione si divide in due canali:

- **Richiesta di carico (HTTP):** la pressione del pulsante genera una chiamata `POST /api/load`. L'adattatore apre una connessione TCP verso `ioportservice`, inoltra la `load_request` a `cargoservice` e restituisce al client la risposta correlata (`load_accepted`, `retrylater` o `load_refused`).

- **Aggiornamento del display (WebSocket):** il browser apre un canale su `/ws/state`. L'adattatore interroga periodicamente `ioportservice` e notifica in tempo reale lo stato di stiva e servizio. In caso di disconnessione, l'interfaccia segnala l'inattendibilità dei dati fino al riallineamento completo alla riconnessione.

6 Deployment

Il deployment implementato mantiene quattro contesti QAK in processi separati sul PC: `ctxcargoservice` su TCP 8020, `ctxcargorobot` su 8021, `ctxioport` su 8022 e `ctxsensor` su 8023.

Il **Raspberry Pi Pico W è un nodo fisico esterno al PC**, con sonar e LED. Legge il codice MicroPython e comunica via Wi-Fi con MQTT sulla porta 1883 del PC.

L'adattatore `web/server.py` espone i servizi HTTP e WebSocket sulla porta 8080. L'interazione con l'interfaccia `web/index.html` si articola su due canali distinti:

- **Gestione delle richieste (HTTP):** ciascuna richiesta `POST` inviata dal client apre una connessione TCP dedicata verso `ioportservice`, garantendo la puntuale correlazione della relativa risposta applicativa (`load_accepted`, `retrylater` o `load_refused`).
- **Aggiornamento dello stato (WebSocket):** il display della IOPort riceve gli aggiornamenti in tempo reale mediante il canale `/ws/state`. L'adattatore interroga periodicamente `ioportservice` ogni 500 ms e inoltra al browser ogni variazione. A ogni connessione o riconnessione, l'adattatore richiede lo stato completo a `ioportservice` e `cargoservice` per riallineare subito lo stato della stiva e l'operatività del servizio.

6.1 Preparazione del Pico W esterno

- **Sonar HC-SR04:** collegato con `TRIG` su `GP15` ed `ECHO` su `GP16`. `ECHO` (5 V) viene ridotto a 3,3 V tramite un partitore di tensione (1 kΩ / 2 kΩ).
- **LED :** connesso al pin `GP17` con resistenza di limitazione da 330 Ω verso massa.

```
mpremote connect COM5 mip install umqtt.simple
mpremote connect COM5 fs cp firmware/config.py :config.py
mpremote connect COM5 fs cp firmware/main.py :main.py
mpremote connect COM5 reset
```

Ogni 250 ms il firmware pubblica su `cargo/picow/sonar` campioni numerati e associati a una sessione (QoS 0, senza retain), mentre riceve i comandi del LED su `cargo/picow/led/set` (QoS 1). Per riallineare il dispositivo a ogni riconnessione, il broker memorizza lo stato desiderato del LED tramite il retain. Al contrario, il servizio a monte scarta eventuali misure retained del sonar, azzerando la continuità delle osservazioni a ogni cambio di sessione o ripristino del collegamento.

6.2 Configurazione e avvio sul PC

I comandi seguenti sono eseguiti da PowerShell:

```
$env:JAVA_HOME = 'C:\Program Files\Java\jdk-17'
.\gradlew.bat test prepareLaunch
```

La sequenza per il deployment con hardware è:

1. Avviare broker e WEnv con il comando seguente. `navigation` aggiunge WEnv al broker e non richiede la rete Docker esterna.

   ```
   docker compose -p cargoservice-sprint2 -f deploy/compose.yaml --profile navigation up -d
   ```

2. Configurare `RobotSmart26` sulla porta 8019:

   ```
   .\scripts\configure-robotsmart.ps1 -RobotSmartRoot C:\percorso\issLab2026\robotsmart26
   ```

   Lo script aggiorna porta e mappa `tf25map.txt`, conservando copie dei file originali. In un altro terminale, nella cartella di RobotSmart26, eseguire `.\gradlew .bat run`.

3. Misurare la distanza con la porta sgombra e fissare D_{FREE} in centimetri.

   ```
   .\scripts\start-system.ps1 -SkipBroker -DFree 25
   ```

   Lo script controlla la disponibilità delle porte e del servizio RobotSmart su 8019, poi avvia quattro JVM e un processo Python in background.

4. Alimentare il Pico W configurato e aprire `http://127.0.0.1:8080`. All'avvio la stiva è vuota, il LED è spento e il sensore non è ancora disponibile. La pagina mostra stiva, barcode, operatività, fase e disponibilità del collegamento.

I valori impostati sono: conferma di presenza e anomalia 3 s, massima lacuna 500 ms, indisponibilità dopo oltre 2 s senza dati affidabili, attesa del container 30 s, marcatura 5 s e timeout di navigazione 120 s.

7 Conclusioni

Con lo Sprint 2 si conclude lo sviluppo del sistema `cargoservice`. Il prototipo finale soddisfa tutti i requisiti concordati con il committente:

- l'automazione completa del ciclo di carico, dal rilevamento del container alla IOPort fino al deposito finale e al rientro del robot in HOME.
- l'integrazione con i dispositivi fisici su Raspberry Pi Pico W (sonar HC-SR04 e LED) via protocollo MQTT.
- il rispetto dei requisiti temporale e delle condizioni di fuori servizio per guasto al sonar o interruzione del flusso dati.
- la realizzazione della Web-GUI per la IOPort, che consente l'invio delle richieste e il monitoraggio in tempo reale dello stato della stiva e del servizio.

Essendo questo lo sprint conclusivo, non sono pianificati ulteriori piani di sviluppo. Il codice sorgente completo, i file di configurazione per il deployment e i test automatizzati sono disponibili nel repository di progetto:

`https://github.com/Edward713/ISS2026-Buttazzi`